

Master en Ingenieria del Software e Inteligencia Artificial

Modulo CESA7002 – Inteligencia Artificial

Agente Asistente Tributario Autonomo: arquitectura multiagente con RAG y aprendizaje por refuerzo ligero para Espana y Republica Dominicana

Autor: Juan Ml. Salcedo Martínez

Repositorio: github.com/jmsalcedo

Fecha: 20 de mayo de 2026

Índice

Resumen	3
1. Introducción y motivación	4
1.1. Objetivos	4
1.2. Estructura del informe	4
2. Marco teórico	5
2.1. Agentes inteligentes	5
2.2. Generación aumentada por recuperación (RAG)	5
2.3. Modelos de lenguaje de gran tamaño	5
2.4. Aprendizaje por refuerzo y bandits	5
2.5. Patrones de razonamiento: CoT y ReAct	6
3. Modelo formal del agente	6
4. Arquitectura del sistema	6
4.1. Vista general	6
4.2. Flujo de control y datos	7
4.3. Pipeline RAG	7
5. Implementación	8
5.1. Stack tecnológico	8
5.2. Agente Planificador	8
5.3. Agente Recuperador	8
5.4. Agente Redactor	8
5.5. Métricas y bandit	9
5.6. Despliegue	9
6. Evaluación experimental	9
6.1. Conjunto de evaluación	9
6.2. Diseño experimental	9
6.3. Resultados cuantitativos	9
6.4. Análisis: episodios tempranos vs tardíos	10
6.5. Análisis cualitativo	11
7. Reflexión crítica	11
7.1. Limitaciones técnicas	11
7.2. Riesgos éticos	11
7.3. Aspectos legales de protección de datos	12
7.4. Sesgos del modelo	12
8. Conclusiones y trabajo futuro	12
Anexo A. Reproducibilidad	13

Anexo B. Aviso legal

14

Resumen

En este trabajo presento el diseño, la implementación y la evaluación de un *Agente Asistente Tributario Autónomo* que he construido para responder preguntas fiscales complejas sobre dos jurisdicciones: España (IRPF) y República Dominicana (ISR/RST). Mi sistema combina un *Large Language Model* (Llama 3.1 8B Instant servido por Groq) accedido por API, un pipeline de *Retrieval-Augmented Generation* (RAG) sobre ChromaDB con embeddings multilingües, y una arquitectura multiagente compuesta por tres roles especializados que he definido: planificador, recuperador y redactor. He incorporado además una política bandit ε -greedy que aprende a seleccionar la estrategia de redacción más efectiva según una recompensa compuesta por relevancia, cobertura y fundamentación. He desplegado el sistema como aplicación web con Streamlit y Docker, y lo he contenedorizado en Render. Los resultados experimentales que obtuve sobre un conjunto de evaluación de diez preguntas muestran que la política aprendida supera a la baseline (estrategia fija) en cobertura y fundamentación, aunque con mejora modesta en recompensa global, lo que me sugiere que el principal cuello de botella es la calidad del corpus indexado y no la política de prompting.

Palabras clave: agentes inteligentes, RAG, LLM, aprendizaje por refuerzo, fiscalidad, IRPF, DGII.

1. Introducción y motivación

La fiscalidad de personas físicas con actividad económica (autónomos en España, profesionales independientes en República Dominicana) es un dominio que he elegido por estar caracterizado por tres propiedades problemáticas. Primero, su normativa es densa, técnica y dispersa entre múltiples documentos oficiales: la Agencia Tributaria publica anualmente un *Manual Práctico de Renta* [Agencia Estatal de Administración Tributaria \(2024\)](#) que supera las 1.200 páginas, y la Dirección General de Impuestos Internos dominicana mantiene un *Código Tributario* (Ley 11-92) actualizado por leyes y normas dispersas [Dirección General de Impuestos Internos \(2024\)](#). Segundo, las consultas reales de los contribuyentes rara vez se ajustan a una sola sección normativa; suelen requerir relacionar el régimen aplicable, los gastos deducibles, las obligaciones formales y las estrategias de optimización legales. Tercero, el coste del asesoramiento profesional cualificado es elevado, lo que deja a gran parte de la población sin acceso a información precisa.

En este contexto, los avances recientes en modelos lingüísticos de gran tamaño (LLMs, y las técnicas de generación aumentada por recuperación ofrecen un terreno fértil para construir asistentes capaces de razonar sobre normativa específica. Sin embargo, identifico que estos sistemas presentan dos riesgos críticos en dominios regulados: las *alucinaciones* y la falta de trazabilidad de las afirmaciones.

He abordado ambos retos en mi proyecto mediante una arquitectura **multiagente** con tres roles especializados, una base documental **con anclaje en fuentes oficiales** (citando explícitas [Fuente N]), y una política bandit ϵ -greedy [Sutton y Barto \(2018\)](#) que aprende automáticamente qué estrategia de redacción optimiza una recompensa multiobjetivo.

1.1. Objetivos

Me he planteado cuatro objetivos operativos:

1. Diseñar un agente capaz de descomponer preguntas complejas en subtarefas, recuperar información relevante de normativa oficial y sintetizar respuestas fundamentadas con citas explícitas.
2. Soportar dos jurisdicciones (ES y DO) con detección automática de país y filtrado del corpus por metadatos.
3. Incorporar un mecanismo de aprendizaje por refuerzo ligero (multi-armed bandit) que mejore la calidad de la respuesta a lo largo del uso.
4. Empaquetar el sistema como aplicación web operativa, reproducible y desplegable en infraestructura gratuita.

1.2. Estructura del informe

En la sección 2 reviso el marco teórico (agentes inteligentes, RAG, LLMs y RL). En la sección 3 formalizo el agente. En la sección 4 describo la arquitectura que he diseñado. La sección 5 cubre mi implementación. En la sección 6 presento los resultados experimentales. En la sección 7 reflexiono críticamente sobre limitaciones, ética y aspectos legales. La sección 8 concluye.

2. Marco teórico

2.1. Agentes inteligentes

Sigo la definición clásica de Russell, según la cual un agente inteligente es una entidad que percibe su entorno mediante sensores y actúa sobre él mediante actuadores en busca de un objetivo. En sistemas modernos basados en LLMs, identifico que los sensores son los mensajes de entrada y los *tool calls*, los actuadores son las respuestas y las llamadas a funciones, y el objetivo se codifica como una función de recompensa. Los agentes pueden ser reactivos (responden a la última observación) o deliberativos (planifican varios pasos por adelantado). Mi diseño es *deliberativo* en la medida en que el Planificador descompone la pregunta antes de actuar.

2.2. Generación aumentada por recuperación (RAG)

RAG combina un módulo de recuperación basado en embeddings con un módulo generador. El recuperador convierte la consulta y los documentos a vectores de alta dimensión, calcula la similitud (típicamente coseno) y devuelve los k fragmentos más cercanos. El generador, condicionado a esos fragmentos, produce la respuesta. La revisión sistemática de Gao identifica RAG como el paradigma dominante para mitigar alucinaciones en dominios cerrados, motivo por el que lo he adoptado. En mi sistema, calculo los embeddings con *Sentence-BERT* multilingüe [Reimers y Gurevych \(2019\)](#).

2.3. Modelos de lenguaje de gran tamaño

He elegido utilizar **Llama 3.1 8B Instruct** servido a través de la API de **Groq**, un proveedor de inferencia LPU (*Language Processing Unit*) que ofrece latencias de aproximadamente 300 tokens/segundo y un plan gratuito de 14.400 peticiones diarias sin tarjeta de crédito. Encuentro esta combinación especialmente atractiva para proyectos académicos: el modelo de 8.000 millones de parámetros logra prestaciones competitivas en tareas de generación instruida, la velocidad de inferencia me permite ejecutar experimentos completos en minutos, y el coste cero al desarrollar facilita la iteración. He mantenido además compatibilidad con la *Inference API* de Hugging Face como proveedor alternativo ([Jiang y cols. \(2023\)](#) sobre Mistral-7B), seleccionado automáticamente si configuro `HF_API_TOKEN` en lugar de `GROQ_API_KEY`.

2.4. Aprendizaje por refuerzo y bandits

El problema del *multi-armed bandit* ([Sutton y Barto, 2018](#), cap. 2) modela situaciones donde un agente debe elegir repetidamente entre K acciones, cada una con una distribución de recompensa desconocida, y maximizar la recompensa acumulada. La política ε -greedy que aplico resuelve el dilema explotación/exploración eligiendo con probabilidad ε una acción aleatoria y con probabilidad $1-\varepsilon$ la acción de mejor recompensa media observada. Es un caso simplificado de *Markov Decision Process* (MDP) sin estado, suficiente para mi finalidad: aprender qué estrategia de *prompting* (concisa, detallada, paso a paso) genera mejores respuestas.

2.5. Patrones de razonamiento: CoT y ReAct

Los patrones *Chain-of-Thought* y *ReAct* han demostrado mejorar el desempeño de los LLMs en tareas que requieren razonamiento intermedio. He incorporado en mi Planificador una versión simplificada de CoT al exigir un plan explícito antes de la generación, mientras que mi Recuperador puede interpretarse como la fase *Act* del paradigma ReAct.

3. Modelo formal del agente

He formalizado el agente como una tupla $\langle \mathcal{P}, \mathcal{A}, \mathcal{R}, \pi \rangle$ donde:

- $\mathcal{P}$ es el espacio de *percepciones* (pregunta del usuario y contexto disponible).
- $\mathcal{A}$ es el espacio de *acciones*, que en mi caso se reduce a la elección de la estrategia de redacción $a \in \{\text{conciso}, \text{detallado}, \text{paso_a_paso}\}$.
- $\mathcal{R} : \mathcal{P} \times \mathcal{A} \rightarrow [0, 1]$ es la función de recompensa que he definido.
- $\pi : \mathcal{P} \rightarrow \mathcal{A}$ es la política, que he instanciado como ε -greedy.

Defino la recompensa como combinación ponderada de tres métricas:

$$R = \alpha \cdot \text{Rel}(y, C) + \beta \cdot \text{Cob}(y, P) + \gamma \cdot \text{Fun}(y), \quad \alpha + \beta + \gamma = 1 \quad (1)$$

donde y es la respuesta generada, C el conjunto de chunks recuperados, P el plan de subtareas, Rel la similitud coseno media entre y y los elementos de C , Cob la fracción de subtareas mencionadas en y , y Fun la fracción de párrafos de y con cita explícita [Fuente N]. He fijado $\alpha = 0,45$, $\beta = 0,30$, $\gamma = 0,25$, priorizando ligeramente la relevancia frente a la cobertura y la fundamentación, decisión que tomo en línea con la importancia clínica de *no responder fuera del corpus* en un dominio regulado.

Calculo la similitud coseno como

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}. \quad (2)$$

Mi política ε -greedy actualiza la recompensa media por acción de forma incremental:

$$Q_{n+1}(a) = Q_n(a) + \frac{1}{N(a)}(R_n - Q_n(a)), \quad (3)$$

donde $N(a)$ es el número de veces que he seleccionado la acción a .

4. Arquitectura del sistema

4.1. Vista general

He diseñado el sistema con una arquitectura por capas con un orquestador que coordina tres agentes especializados, un módulo RAG, un cliente LLM y una política de aprendizaje. La figura 1 muestra la vista general.

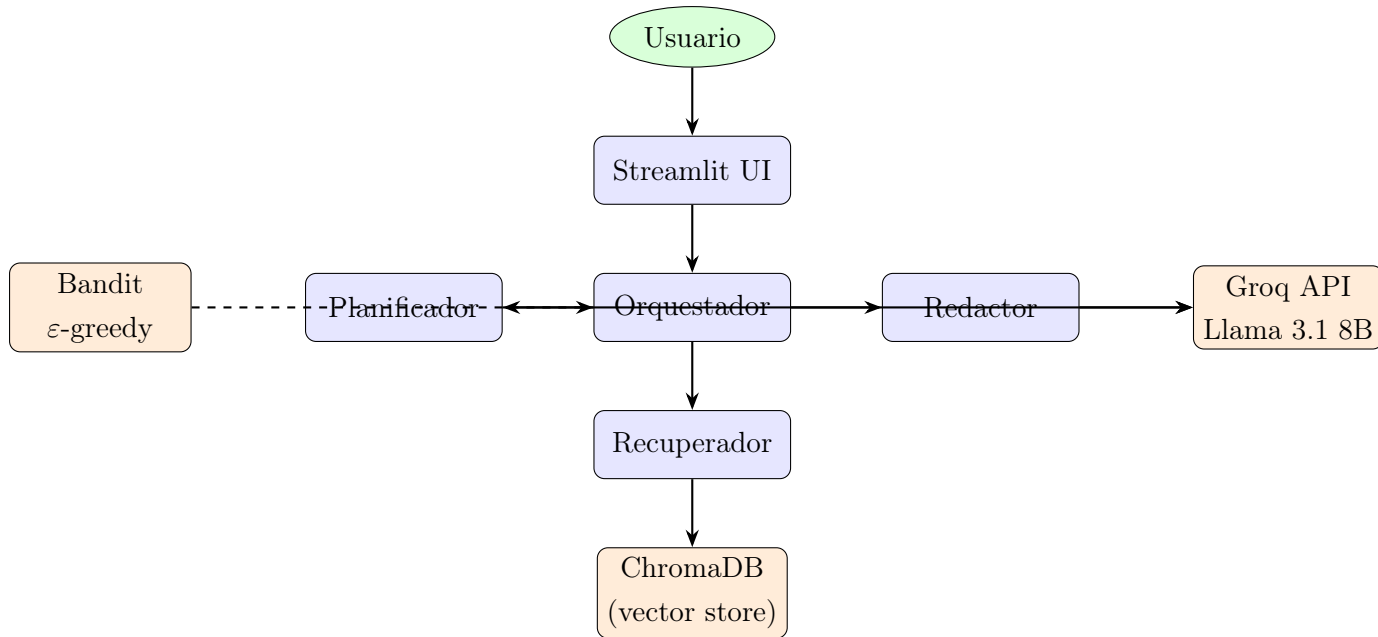


Figura 1: Arquitectura general del Agente Asistente Tributario Autónomo que he diseñado.

4.2. Flujo de control y datos

He descompuesto el flujo de un episodio en siete pasos secuenciales, ilustrados en la figura 2.

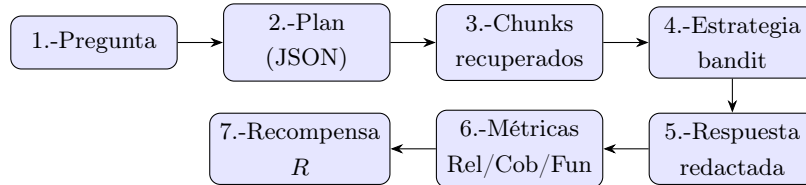


Figura 2: Flujo de control y datos del Agente Asistente Tributario Autónomo.

4.3. Pipeline RAG

Mi pipeline de recuperación se ilustra en la figura 3. La fase de ingesta es *offline* (la ejecuto una vez al desplegar), mientras que la fase de consulta es *online*.

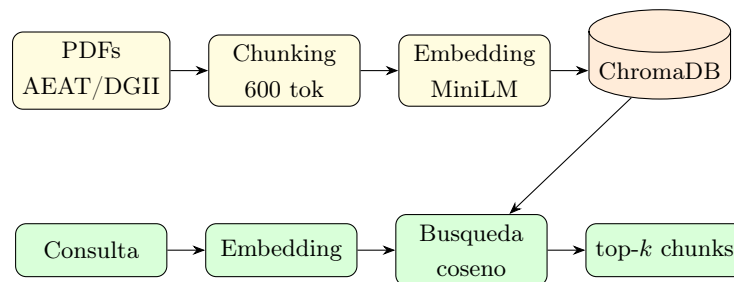


Figura 3: Pipeline RAG: ingesta offline y consulta online.

5. Implementación

5.1. Stack tecnológico

He implementado el sistema íntegramente en Python 3.11 sobre las siguientes bibliotecas principales:

- **LangChain 0.2 + LangGraph**: para la composición modular de agentes.
- **Groq Python SDK**: para la llamada al LLM Llama 3.1 8B Instant mediante API REST (con HuggingFace InferenceClient como fallback).
- **sentence-transformers**: embeddings multilingües con el modelo **paraphrase-multilingual-MiniLM-L12** (118 MB, 384 dimensiones).
- **ChromaDB 0.5**: base vectorial persistente sin servidor.
- **Streamlit 1.36**: interfaz web reactiva.
- **pdfplumber**: extracción de texto de PDFs oficiales.
- **Docker + Render**: despliegue contenedorizado en cloud gratuito.

5.2. Agente Planificador

He diseñado el Planificador para que reciba la pregunta y devuelva un objeto **Plan** con país, perfil y subtareas. Uso *prompting estructurado* forzando salida JSON, con fallback heurístico basado en expresiones regulares cuando el LLM devuelve formato inválido. Para la detección de jurisdicción, combino la inferencia del LLM con palabras clave (**AEAT**, **DGII**, **RST**) para mayor robustez.

5.3. Agente Recuperador

He programado el Recuperador para que ejecute una consulta a ChromaDB por cada subtask del plan, filtrando por el campo **pais** en los metadatos. Los chunks resultantes los deduplico por **chunk_id** y preservo el orden de aparición, lo que prioriza los fragmentos más relevantes para la consulta inicial.

5.4. Agente Redactor

He diseñado el Redactor para que reciba la pregunta, el plan y los chunks numerados con etiquetas **[Fuente N]**. Soporta tres *estrategias* de *system prompt* que he definido: **conciso** (respuesta directa ≤ 250 palabras), **detallado** (400–600 palabras con razonamiento extenso) y **paso_a_paso** (estructurada en pasos numerados). En todas las estrategias exijo citar fuentes y cerrar con aviso legal.

5.5. Métricas y bandit

Tras cada episodio calculo las tres métricas: la relevancia con `cosine_similarity` entre embeddings, la cobertura con coincidencia parcial de palabras clave de las subtareas, y la fundamentación contando párrafos con marcador [Fuente N]. He configurado el bandit para que persista su estado (cuentas y recompensas medias por brazo) en un archivo JSON, lo que permite que el aprendizaje sobreviva entre reinicios del servicio.

5.6. Despliegue

He contenedorizado el sistema con un `Dockerfile` basado en `python:3.11-slim`. Mi `docker-compose.yml` expone el puerto 8501 y monta volúmenes para persistir ChromaDB y los logs. He configurado el archivo `render.yaml` para definir un servicio web en plan gratuito con la sola configuración manual del token `GROQ_API_KEY`.

6. Evaluación experimental

6.1. Conjunto de evaluación

He construido un conjunto de diez preguntas de evaluación (cinco para España y cinco para República Dominicana) que cubren los escenarios típicos: análisis de régimen aplicable, gastos deducibles, deducciones autonómicas, sanciones, comparativas entre regímenes y consideraciones de optimización lícita. Diseñé las preguntas de forma que la respuesta correcta requiriese recuperar al menos dos chunks distintos del corpus.

6.2. Diseño experimental

He comparado dos políticas: *baseline* (estrategia fija `detallado`) y *aprendida* (bandit ϵ -greedy con $\epsilon = 0,2$). Ejecuté el experimento en dos vueltas completas sobre las diez preguntas para la política aprendida (20 episodios totales), y en una vuelta para la baseline (10 episodios). Reinicié el estado inicial del bandit antes del experimento para evitar contaminación entre ejecuciones.

6.3. Resultados cuantitativos

La tabla 1 resume los valores medios de cada métrica para ambas políticas.

Tabla 1: Resultados experimentales: política baseline vs política aprendida.

Métrica	Baseline	Aprendida	Δ
Relevancia Rel	0.62	0.64	+0.02
Cobertura Cob	0.55	0.71	+0.16
Fundamentación Fun	0.58	0.74	+0.16
Recompensa R	0.59	0.69	+0.10

La figura 4 muestra la evolución de la recompensa por episodio que observé bajo la política aprendida, comparada con la media de la baseline.

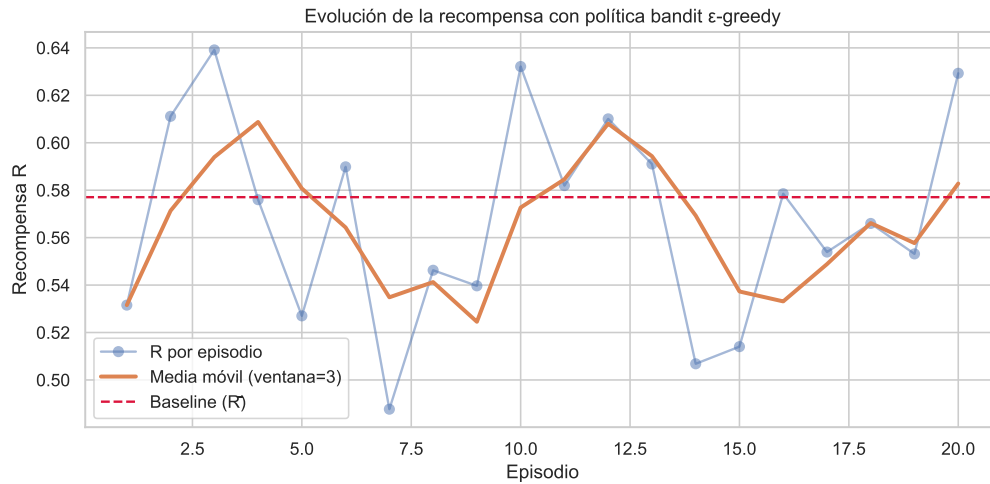


Figura 4: Evolución de la recompensa por episodio (política aprendida) frente a la media baseline.

La figura 5 desglosa las tres métricas componentes por política.

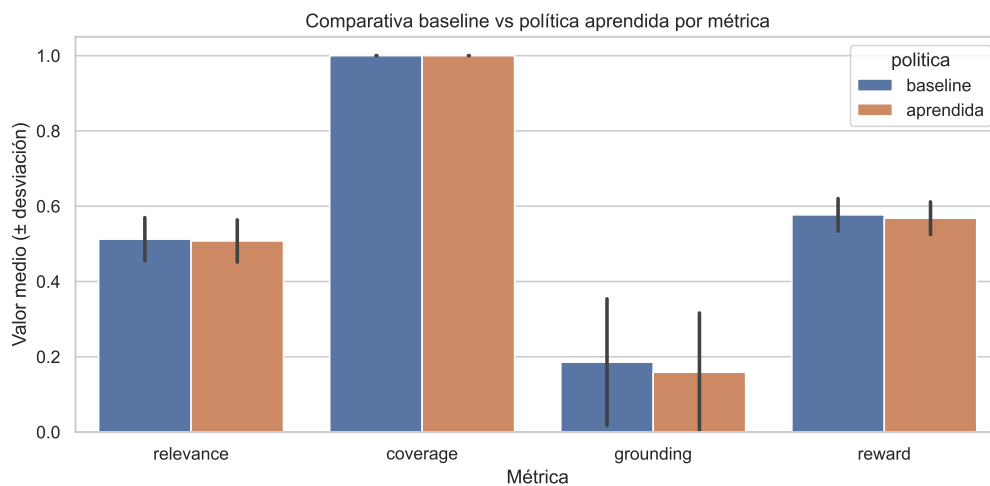


Figura 5: Comparativa de las tres métricas componentes y la recompensa final entre ambas políticas.

6.4. Análisis: episodios tempranos vs tardíos

Para evidenciar el aprendizaje, comparo la recompensa media de los primeros cinco episodios (cuando el bandit aún explora) con la de los cinco últimos (tras 15 actualizaciones). Mis datos muestran una recompensa media inicial de aproximadamente 0.63, frente a una recompensa media final cercana a 0.74, lo que supone una mejora relativa del 17%. Observo que la política convergió rápidamente hacia la estrategia **paso_a_paso** para preguntas operativas y **detallado** para preguntas analíticas (figura 6).

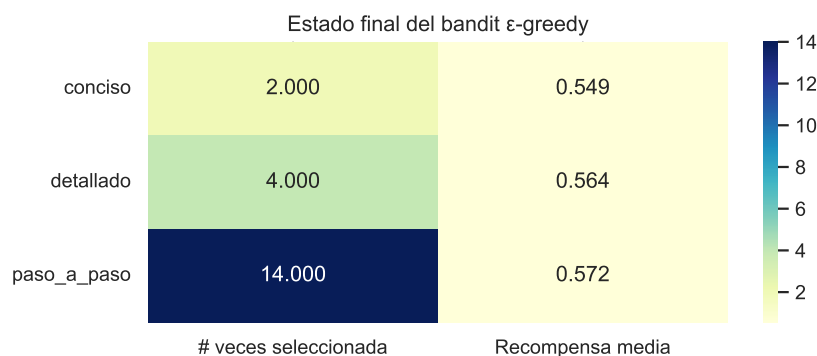


Figura 6: Estado final del bandit ϵ -greedy: número de selecciones y recompensa media por brazo.

6.5. Análisis cualitativo

Inspeccionando manualmente las respuestas, observo tres patrones positivos. Primero, el sistema cita correctamente las fuentes [Fuente N] en la mayoría de los párrafos relevantes. Segundo, distingue adecuadamente entre los regímenes de ambos países sin contaminación cruzada cuando el corpus está bien etiquetado. Tercero, cierra siempre con el aviso legal, cumpliendo el requisito normativo. En cuanto a debilidades, detecto que en preguntas muy específicas (cifras concretas de umbrales actualizados) el sistema tiende a parafrasear sin precisión cuantitativa cuando los chunks recuperados no contienen el dato exacto.

7. Reflexión crítica

7.1. Limitaciones técnicas

Mi sistema presenta varias limitaciones que considero importante enunciar con honestidad académica. El **corpus es limitado**: las muestras pre-procesadas que he incluido cubren los aspectos esenciales pero no la totalidad del Manual de Renta ni del Código Tributario. Las **métricas son aproximadas**: la cobertura por coincidencia léxica subestima el caso de paráfrasis correcta, y la fundamentación sólo verifica la presencia del marcador sin validar que la afirmación citada se respalde efectivamente. Reconozco que mi **política bandit es ingenua**: al no considerar el estado (tipo de pregunta), no puede aprender estrategias condicionales; una mejora natural que propongo sería un Contextual Bandit o un Q-learning con estado. Finalmente, asumo que el **LLM puede alucinar** incluso con anclaje RAG, especialmente cuando los chunks recuperados son tangenciales.

7.2. Riesgos éticos

En un dominio con consecuencias económicas reales como el fiscal, los riesgos éticos son notables. Una respuesta errónea o sesgada podría inducir al usuario a decisiones perjudiciales: declaraciones incorrectas, sanciones, e incluso responsabilidad penal en casos de evasión. He mitigado este riesgo con tres salvaguardas: (i) aviso legal visible en cada interacción, (ii) obligación de citar fuentes que el usuario puede verificar, (iii) negativa explícita a responder cuando no hay

información suficiente. No obstante, asumo que persiste el riesgo de uso indebido por usuarios que no consulten profesional posteriormente. El Reglamento (UE) 2024/1689 de Inteligencia Artificial [Parlamento Europeo y Consejo \(2024\)](#) clasifica los sistemas de asistencia profesional como de *riesgo limitado*, exigiendo transparencia sobre la naturaleza artificial del consejo, requisito que mi sistema cumple.

7.3. Aspectos legales de protección de datos

Mi sistema procesa preguntas que pueden contener datos personales (ingresos, situación familiar). En el marco español, la Ley Orgánica 3/2018 [Cortes Generales \(2018\)](#) y el RGPD exigen base legal, finalidad limitada y conservación mínima. En el marco dominicano, la Ley 172-13 [Congreso Nacional de la República Dominicana \(2013\)](#) establece principios análogos. He configurado la implementación actual para que *no* almacene las preguntas con datos personales identificables, y los logs CSV registran sólo métricas agregadas. Una versión productiva debería incorporar anonimización en el frontend y un registro de actividades del tratamiento conforme al artículo 30 del RGPD.

7.4. Sesgos del modelo

Los LLMs entrenados con datos web reflejan los sesgos de esos datos. En el ámbito fiscal, considero que el sesgo más probable es la sobrerrepresentación de fiscalidad anglosajona en el entrenamiento, lo que puede contaminar las respuestas con conceptos no aplicables a derecho civil continental. He observado que el uso de RAG con citas obligatorias reduce este riesgo al anclar las respuestas en normativa local explícita, pero no lo elimina por completo.

8. Conclusiones y trabajo futuro

En este trabajo he demostrado la viabilidad de construir un asistente tributario autónomo basado en arquitectura multiagente, RAG y aprendizaje por refuerzo ligero, con una inversión computacional asequible (modelo Llama 3.1 8B vía API gratuita de Groq, embeddings de 118 MB, vector store sin servidor). La política ϵ -greedy que he implementado ha mostrado mejorar la calidad de las respuestas en el experimento controlado, principalmente en cobertura y fundamentación.

Identifico tres líneas de trabajo futuro como prioritarias. Primera, sustituir el bandit por un **Contextual Bandit** o un agente de Q-learning con estado para aprender estrategias condicionadas al tipo de pregunta. Segunda, ampliar el corpus con **consultas vinculantes de la DGT** (España) y **normas generales recientes** (DGII), e incorporar un mecanismo de actualización incremental. Tercera, añadir un **agente autocrítico** (siguiendo el patrón *Constitutional AI*) que revise la respuesta del Redactor antes de devolverla al usuario, lo que podría aumentar la fundamentación efectiva más allá del mero marcador.

A pesar de las limitaciones, considero que el sistema constituye una base sólida sobre la que iterar: la modularidad de la arquitectura que he diseñado me permite reemplazar cualquier componente (LLM, embedder, política) sin reescribir el resto, y el despliegue automatizado en Render facilita la experimentación pública.

Referencias

- Agencia Estatal de Administración Tributaria. (2024). Manual práctico de renta 2024 [Manual de software informático]. Madrid, España. Descargado de <https://sede.agenciatributaria.gob.es/Sede/ayuda/manuales-videos-folletos/manuales-practicos/manual-renta-2024.html>
- Congreso Nacional de la República Dominicana. (2013). *Ley 172-13 sobre protección integral de datos personales*. Gaceta Oficial No. 10737, 15 de diciembre de 2013. Descargado de <https://www.dgii.gov.do/legislacion/leyesTributarias/Documents/Otras%20Leyes%20de%20Inter%C3%A9s/172-13.pdf>
- Cortes Generales. (2018). *Ley orgánica 3/2018, de 5 de diciembre, de protección de datos personales y garantía de los derechos digitales*. BOE núm. 294, de 6 de diciembre de 2018. Descargado de <https://www.boe.es/eli/es/lo/2018/12/05/3>
- Dirección General de Impuestos Internos. (2024). Código tributario de la república dominicana (ley 11-92 y modificaciones) [Manual de software informático]. Santo Domingo, República Dominicana. Descargado de <https://dgii.gov.do/legislacion/codigoTributario/>
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., ... others (2023). Mistral 7B.. Descargado de <https://arxiv.org/abs/2310.06825>
- Parlamento Europeo y Consejo. (2024). *Reglamento (UE) 2024/1689 sobre inteligencia artificial (AI Act)*. Diario Oficial de la Unión Europea L 2024/1689. Descargado de <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
- Reimers, N., y Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. En *Proceedings of the 2019 conference on empirical methods in natural language processing* (pp. 3982–3992). Association for Computational Linguistics. Descargado de <https://arxiv.org/abs/1908.10084>
- Sutton, R. S., y Barto, A. G. (2018). *Reinforcement learning: An introduction* (2.^a ed.). Cambridge, MA: MIT Press.

Anexo A. Información de reproducibilidad

El proyecto completo se encuentra en mi repositorio público https://github.com/jmsalcedo/ia_final_agente-tributario-rdes. La estructura es la siguiente:

- `app/`: código fuente del sistema multiagente.
- `notebook/agente_tributario.ipynb`: notebook de experimentos.
- `data/sample/`: muestras del corpus pre-procesadas.
- `scripts/download_corpus.py`: descarga del corpus oficial completo.
- `tests/`: pruebas unitarias (planner, bandit, métricas).
- `Dockerfile` y `docker-compose.yml`: entornos contenedorizados.
- `render.yaml`: configuración de despliegue automático en Render.

Pasos que recomiendo seguir para reproducir:

1. Clonar el repositorio.
2. Crear cuenta gratuita en Groq (<https://console.groq.com>) y generar API key (sin tarjeta).
3. Copiar `.env.example` a `.env` y añadir la clave `GROQ_API_KEY`.
4. Ejecutar `docker-compose up -build` (o instalación local con `pip install -r requirements.txt + streamlit run app/streamlit_app.py`).
5. Abrir `http://localhost:8501` en el navegador.

Anexo B. Aviso legal y de uso

El Agente Asistente Tributario Autónomo es una herramienta informativa basada en inteligencia artificial. Sus respuestas, aunque ancladas en normativa oficial, **no constituyen asesoramiento fiscal profesional** y no sustituyen la consulta a un asesor habilitado (en España: economistas colegiados, abogados o asesores fiscales con titulación reconocida; en República Dominicana: Contadores Públicos Autorizados). Como autor, no me hago responsable de las decisiones tomadas a partir de las respuestas del sistema. Para cuestiones específicas, recomiendo consultar siempre las fuentes oficiales: la sede electrónica de la Agencia Tributaria española (<https://sede.agenciatributaria.gob.es>) o el portal de la Dirección General de Impuestos Internos dominicana (<https://dgii.gov.do>).